

STACKBURGER

Aplicación Móvil · Delivery & Takeaway

DOCUMENTACIÓN TÉCNICA

Entrega 3 — Producto final escalado

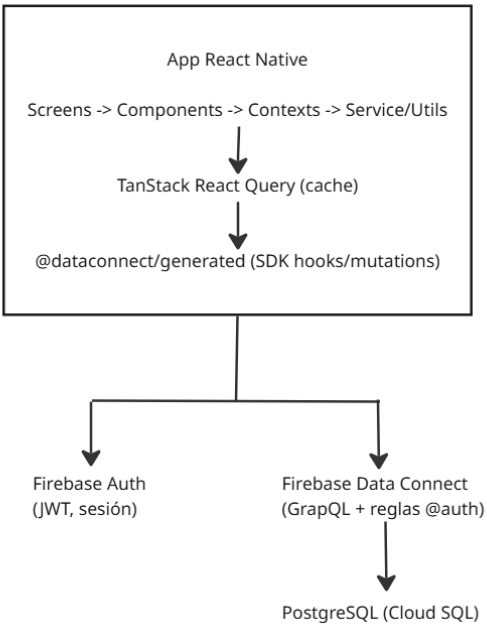
Este documento detalla las decisiones tecnológicas tomadas para la construcción del producto final escalado de StackBurger: stack tecnológico, arquitectura general, pantallas principales, API y servicios, diagramas de secuencias (flujos principales), roles y permisos, seguridad y manejo de los datos, performance y optimizaciones

Proyecto StackBurger App	Framework React Native 0.85.3	Plataforma Android (API 24+)	Node.js ≥ 22.11.0
------------------------------------	---	--	-----------------------------

1. Stack tecnológico

Operación	SDK	Uso
App móvil	React Native	0.85.3
UI	React	19.2.3
Lenguaje	TypeScript	5.8.3
Navegación	React Navigation	Stack Navigator v7
Auth	Firebase Auth	Email/contraseña
Datos	Firebase Data Connect	GraphQL → PostgreSQL
Cache servidor	TanStack React Query	v5 + @tanstack-query-firebase/react
Estado local	Context API + useReducer	Carrito, auth, sucursal
Persistencia sesión	AsyncStorage	vía getReactNativePersistence
Node.js	≥ 22.11.0	

2. Arquitectura general



3. Pantallas principales

Pantalla	Rol	Funcion
Login / Register	Público	Auth
Home	Invitado / Cliente	Catálogo por sucursal
Detail	Invitado / Cliente	Personalización producto
Cart / Checkout	Cliente	Pedido
Orders	Cliente	Historial
AdminProducts	Admin	Productos + precios_sucursal
AdminOrders	Admin	Gestión pedidos
AdminStatistics	Admin	Rankings, gráficos por sucursal
AdminOrderDetail	Admin	Detalle + cambio estado

4. API y servicios

4.1 Firebase authentication

Operación	SDK	Uso
Registro	createUserWithEmailAndPassword	Alta de cliente
Login	signInWithEmailAndPassword	Inicio de sesión
Logout	signOut	Cierre de sesión
Estado sesión	onAuthStateChanged	Restaurar sesión al abrir app
Persistencia	initializeAuth + AsyncStorage	Sesión entre reinicios

Tras login, el token JWT se envía automáticamente a Data Connect en operaciones @auth(level: USER).

4.2 Firebase Data Connect – Modelo de datos (PostgreSQL)

Enums

Enum	Valores
RolUsuario	CLIENTE, ADMIN

CategoriaProducto	HAMBURGUESAS, PAPAS, BEBIDAS, POSTRES
EstadoPedido	PREPARING, COMPLETED, CANCELLED
ModalidadEntrega	DELIVERY, TAKEAWAY
EstadoProductoSucursal	ACTIVO, SIN_STOCK, INEXISTENTE

Tablas / Entidades

Usuario

Campo	Tipo	Notas
id	UUID	PK autogenerada
nombreCompleto	String	
email	String	Unique
authUId	String	Unique, vinculado a Firebase Auth
passwordHash	String?	Legacy/seed; no usado en runtime app
rol	RolUsuario	Default CLIENTE
createdAt	Timestamp	Auto

Producto

Campo	Tipo	Notas
id	UUID	PK
nombre	String	
descripcion	String	
ingredientes	String	
precio	Int	Precio base / fallback
categoria	CategoriaProducto	
imagenUrl	String	URL externa
activo	Boolean	Soft-delete
createdAt	Timestamp	

Sucursal

Campo	Tipo	Notas
-------	------	-------

<code>id</code>	UUID	PK
<code>nombre</code>	String	Unique
<code>direccion</code>	String	Unique

ProductoSucursal (precio y stock de cada producto por sucursal)

Campo	Tipo	Notas
<code>id</code>	UUID	PK
<code>producto</code>	→ Producto	FK
<code>sucursal</code>	→ Sucursal	FK
<code>precio</code>	Int	
<code>estado</code>	EstadoProductoSucursal	

Pedido

Campo	Tipo
<code>id</code>	UUID
<code>codigo</code>	String
<code>usuario</code>	→ Usuario
<code>sucursal</code>	→ Sucursal
<code>estado</code>	EstadoPedido
<code>subtotal, costoEnvio, total</code>	Int
<code>modalidadEntrega</code>	ModalidadEntrega
<code>nombreContacto, telefonoContacto</code>	String
<code>direccion</code>	String?
<code>createdAt</code>	Timestamp

PedidoItem

Campo	Tipo
<code>pedido</code>	→ Pedido
<code>producto</code>	→ Producto?
<code>nombreProducto</code>	String
<code>cantidad</code>	Int
<code>precioUnitario, precioTotalLinea</code>	Int

<code>extraMedallon/Cheddar/Panceta/Cebolla</code>	Int
<code>notas</code>	String?

Relaciones

Usuario 1—N Pedido N—1 Sucursal
 Producto 1—N ProductoSucursal N—1 Sucursal
 Pedido 1—N PedidoItem N—1 Producto (opcional)

4.3 Queries (GraphQL)

Query	Auth	Descripción
<code>ListProductosActivos</code>	PUBLIC	Catálogo global (legacy/fallback)
<code>ListProductosPorSucursal (\$sucursalId)</code>	PUBLIC	Productos + precio/estado por sucursal
<code>ListProductosAdmin</code>	USER	Admin – todos los productos
<code>ListProductoSucursalByProducto (\$productoId)</code>	USER	Precios del producto en cada sucursal
<code>ListSucursales</code>	PUBLIC	Listado de sucursales
<code>ListMyPedidos</code>	USER	Pedidos del usuario (auth.uid)
<code>ListPedidosAdmin</code>	USER	Todos los pedidos (admin)
<code>GetMe</code>	USER	Perfil + rol del usuario logueado
<code>GetProductoById (\$id)</code>	PUBLIC	Detalle de producto

4.4 Mutations (GraphQL)

Mutation	Auth	Descripción
<code>CreateUsuarioProfile</code>	USER	Crea fila en Usuario con authUid
<code>LinkMyAccount</code>	USER	Vincula cuenta Auth con usuario seed por email
<code>CreateProducto / UpdateProducto / DeleteProducto</code>	USER	CRUD productos (delete = activo: false)
<code>CreateProductoSucursal / UpdateProductoSucursal</code>	USER	Precio/estado por sucursal
<code>CreateSucursal</code>	USER	Alta de sucursal
<code>CreatePedido</code>	USER	Crea pedido con sucursalId

CreatePedidoItem	USER	Ítems del pedido
UpdatePedidoEstado	USER	Cambio de estado (admin)

4.5 Reglas de seguridad

Data Connect – niveles @auth

Nivel	Significado
PUBLIC	Sin login (catálogo)
USER	Requiere JWT válido de Firebase Auth

Filtrado por usuario:

- ListMyPedidos: usuario.authUid == auth.uid
- GetMe: authUid == auth.uid
- CreateUsuarioProfile: authUid_expr: "auth.uid"

Limitación (deuda técnica): mutaciones admin (CreateProducto, ListPedidosAdmin, etc.) usan @auth(level: USER) sin filtrar por rol: ADMIN en GraphQL. La restricción admin se aplica en la app (useRequireAdmin), no en el conector server-side.

Firebase Storage (storage.rules)

```

/productos/{productId}/{fileName}
read: true (público)
write: request.auth != null
      && size < 5 MB
      && contentType image/*

```

Para las imagenes se usan URLs en imagenUrl.

Configuración local

- firebaseConfig.ts, .firebaserc, dataconnect/ → **gitignored** (no en GitHub)
- Plantillas .example para setup local

4.6 Servicios en la App

Módulo	Responsabilidad
<u>orderService.ts</u>	Orquesta CreatePedido + CreatePedidoItem
firebaseMappers.ts	DB ↔ modelos UI (Burger, Order, AdminOrder)
productQueryCache.ts	Cache/sync productos admin y activos
orderQueryCache.ts	Refresh pedidos

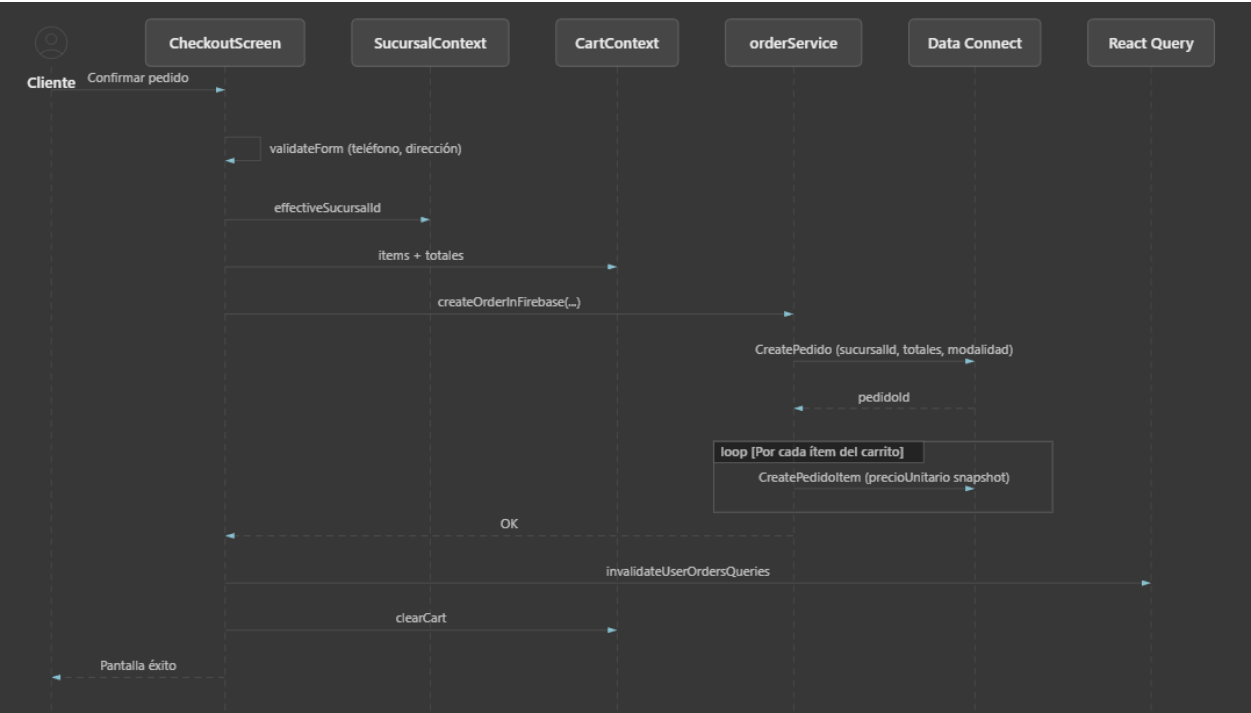
<code>sucursalQueryCache.ts</code>	Refresh sucursales
<code>queryInvalidation.ts</code>	Invalidación selectiva de queries

5. Diagramas de secuencia – Flujos principales

5.1 Login + resolución de perfil y rol



5.2 Confirmación de pedido (cliente + sucursal)



6. Roles y permisos

Funcionalidad	Invitado	Cliente	Admin
Ver catálogo / sucursales	SÍ	SÍ	SÍ
Elegir sucursal	NO	SÍ	N/A
Agregar al carrito / pedir	NO	SÍ	SÍ
Historial propio	NO	SÍ	SÍ
CRUD productos	NO	NO	SÍ
Precios por sucursal	NO	NO	SÍ
Ver todos los pedidos	NO	NO	SÍ
Cambiar estado pedido	NO	NO	SÍ
Estadísticas (AdminStatisticsScreen)	NO	NO	SÍ

7. Seguridad y manejo de datos

7.1 Datos del usuario almacenados

Dónde	Qué se guarda
Firestore Auth	email, UID, hash de contraseña (Firestore)
PostgreSQL – Usuario	nombreCompleto, email, authUid, rol
PostgreSQL – Pedido	nombreContacto, telefonoContacto, direccion (por pedido)
AsyncStorage (local)	Token/sesión Firestore Auth (persistencia)
App (memoria)	Carrito, sucursal seleccionada (no persistido en DB hasta checkout)

No se almacena en la app: contraseña en texto plano, tarjetas, datos de pago.

7.2 Protección de datos

Medida	Implementación
Transporte	HTTPS (Firestore SDK)
Autenticación	Firestore Auth JWT
Autorización app	useRequireAdmin, AuthGate, modo invitado
Autorización backend	@auth en queries/mutations
Secretos	.gitignore para config Firestore
Soft-delete productos	activo: false (no borrado físico)
Snapshot pedidos	PedidoItem guarda nombre y precio histórico

7.3 Política de sesiones y tokens

Aspecto	Comportamiento
Token	JWT emitido por Firestore Auth
Persistencia	getReactNativePersistence(AsyncStorage)
Renovación	Automática por SDK Firestore (refresh token)
Expiración	Gestionada por Firestore (~1 h ID token; refresh transparente)
Cierre sesión	signOut(auth) → limpia AuthContext.user
Sesión inválida	Si getMe falla tras login → signOut automático

La app no implementa expiración manual ni refresh custom: delega en Firebase Auth.

7.4 Validaciones de entrada

Cliente-Side (app)

Pantalla/Flujo	Validación
Login	Email y password no vacíos
Register	Nombre, email, password ≥ 6 , confirmación coincide
Checkout	Teléfono ≥ 8 dígitos; dirección ≥ 5 chars si delivery
Checkout	Usuario autenticado; sucursal confirmada
Admin productos	Nombre, descripción, precio numérico > 0
Precios sucursal	Precio entero > 0 por fila
Carrito invitado	Modal login requerido
Cambio sucursal	Alerta + vaciado carrito si hay ítems
Producto sin stock	Bloqueo en BurgerCard

Server-Side (PostgreSQL + GraphQL)

Validación	Mecanismo
Tipos	Schema GraphQL (Int!, String!, UUID!)
Unicidad	@unique en email, codigo, authUid
Auth	@auth(level: USER) rechaza sin token
FK	Relaciones Usuario/Pedido/Sucursal/Producto
Enums	Solo valores definidos (estado, rol, categoría)

8. Performance y optimizaciones

8.1 Estrategias de caché

Parámetro	Valor	Efecto
staleTime	60 s	No refetch agresivo en listas

<code>gcTime</code>	10 min	Retiene cache en memoria
<code>refetchOnWindowFocus</code>	false	Evita refetch al volver a app (mobile)
<code>refetchOnReconnect</code>	true	Actualiza al recuperar red

Cache manual

- **preserveProductListOrder:** mantiene orden visual al sincronizar admin
- **patchAdminProductInCache:** actualización in-place tras editar (sin reordenar)
- **syncProductCachesAfterEdit:** sincroniza admin + catálogo activo
- **Invalidación selectiva:** solo queries afectadas (pedidos, productos, sucursales)

8.2 Optimizaciones UI

Técnica	Ubicación
<code>React.memo</code>	BurgerCard, HomeHeader, OrderCard, cards admin
<code>useMemo / useCallback</code>	Home, filtros, mappers de listas
<code>FLAT_LIST_PERF_PROPS</code>	initialNumToRender: 8, windowSize: 5, batching
<code>removeClippedSubviews</code>	Android en FlatLists
Header desacoplado del carrito	useCartItemCount vs contexto completo
<code>fadeDuration={0}</code> en imágenes	Cards (evita parpadeo)
Sin refetch on focus agresivo	Admin products (pull-to-refresh manual)